

R102 - TP n°1 : Réponses - Capture et analyse ICMP

Question 1 : Construction du réseau

Architecture réseau à créer :

- **RT-Box2** (Passerelle) :
 - Interface Bridge (eth0) : @IP 172.31.xx.xx/20 (réseau département)
 - Interface Interne (eth1) : @IP 192.31.25.1/24 (réseau interne)
- **MV Ubuntu Desktop** : carte réseau interne
- **MV Kali** : carte réseau interne
- **MV Windows 10** : carte réseau interne
- **MV Mac Sierra** : carte réseau interne

Toutes les MV internes recevront une @IP 192.31.25.x via DHCP de la RT-Box2.

Question 2 : Configuration IPv4

Commandes à utiliser :

RT-Box2, Ubuntu, Kali :

```
ip a
```

Mac :

```
ifconfig
```

Windows :

```
ipconfig
```

Résultats attendus :

- RT-Box2 : 172.31.xx.xx (eth0) et 192.31.25.1 (eth1)
- Ubuntu/Kali/Windows/Mac : 192.31.25.x/24

Question 3 : Test de connectivité

```
ping 192.31.25.1
```

Toutes les machines doivent pouvoir pinguer la passerelle.

Question 4 : Capture et analyse d'un ping Ubuntu

Commande Ubuntu :

```
ping -c1 192.31.25.1
```

Encapsulation de la trame ICMP Echo Request :

EN-TÊTE ETHERNET (14 octets)
@MAC Destination (6 octets) : @MAC RT-Box2 @MAC Source (6 octets) : @MAC Ubuntu Type (2 octets) : 0x0800 (IPv4)
EN-TÊTE IPv4 (20 octets minimum)
Version: 4 IHL: 5 TOS: 0x00 Longueur totale : 84 octets Identification : 0xFFFF Flags: 0x4000 (DF=1, MF=0) Fragment Offset: 0 TTL: 64 Protocole: 1 (ICMP) Checksum: 0xFFFF @IP Source : 192.31.25.x (Ubuntu) @IP Destination : 192.31.25.1 (RT-Box2)
EN-TÊTE ICMP (8 octets)
Type: 8 (Echo Request) Code: 0 Checksum: 0xFFFF Identifiant: 0xFFFF Sequence Number: 0x0001
DONNÉES ICMP (56 octets)
Timestamp (8 octets) Data (48 octets) : 0x10 11 12 13 ... 37

Filtre Wireshark : icmp

Question 5 : Comparaison des pings selon les OS

Tableau comparatif des différences

OS	TTL par défaut	Timestamp	Taille données ICMP	Contenu des données
Ubuntu	64	Oui	48 octets (56-8)	0x10 11 12 13 ... 37 (40 octets de pattern)
Kali	64	Oui	48 octets	0x10 11 12 13 ... 37 (40 octets de pattern)
Mac	64	Oui	56 octets	0x08 09 0a ... 37 (48 octets de pattern)
Windows	128	Non	32 octets	"abcdefghijklmnopqrstuvwabcdefghi" (ASCII)

Commandes utilisées :

- Ubuntu/Kali : `ping -c1 192.31.25.1`
- Mac : `ping -c1 192.31.25.1`
- Windows : `ping -n 1 192.31.25.1`

Question 6 : Intérêt de l'observation

Utilité : OS Fingerprinting (identification de l'OS)

Ces différences permettent de déterminer l'OS qui a envoyé un ping :

1. **TTL :**
 - a. 64 → Linux/Unix/Mac
 - b. 128 → Windows
 - c. 255 → Certains équipements réseau
2. **Présence de timestamp :**
 - a. Oui → Linux/Unix/Mac
 - b. Non → Windows
3. **Contenu des données :**
 - a. Pattern hexadécimal incrémental → Linux/Mac
 - b. Alphabet ASCII répété → Windows
 - c. Taille différente selon l'OS

Application pratique : En analysant le trafic réseau, un administrateur peut identifier les types de machines présentes sur son réseau sans accès direct.

Question 7 : CTF - Message caché

Procédure :

1. Ouvrir `ctf.pcap` avec Wireshark
2. Appliquer le filtre : `icmp`
3. Observer les données ICMP de chaque paquet
4. Extraire le contenu du champ "Data"
5. Convertir les valeurs hexadécimales en ASCII

Le message caché se trouve dans les données ICMP des paquets Echo Request/Reply.

Pour extraire le message :

- Clic droit sur un paquet ICMP → Follow → UDP Stream (ou analyser les data manuellement)
- Les données ICMP contiennent des caractères ASCII formant un message

Question 8 : Test des intervalles de ping

Commandes à tester :

```
# Ping toutes les 2 secondes
ping -i2 192.31.25.1
# Ping toutes les 0.1 secondes (nécessite root)
sudo ping -i0.1 192.31.25.1
# Ping le plus rapide possible (nécessite root)
sudo ping -i0.001 192.31.25.1
# ou encore plus rapide :
sudo ping -f 192.31.25.1 # flood ping
```

Valeur minimale d'intervalle :

- Sans privilèges root : 0.2 secondes minimums
- Avec root : jusqu'à 0.001 seconde ou mode flood (-f)

Pour arrêter : Ctrl+C

Question 9 : Test sur autres OS

Ubuntu/Kali

```
ping -i2 192.31.25.1
sudo ping -i0.1 192.31.25.1
```

✅ Fonctionne

Mac

```
ping -i 2 192.31.25.1
```

✅ Fonctionne (syntaxe similaire)

Windows

```
ping -t 192.31.25.1
```

✗ ATTENTION : Sous Windows, -i définit le TTL, pas l'intervalle !

Options Windows équivalentes :

- ping -t : ping continu jusqu'à Ctrl+C
- ping -n <count> : nombre d'échos

- `-w <timeout>` : timeout en ms

Pas d'option native pour l'intervalle sous Windows. Il faut utiliser PowerShell :

```
while ($true) { ping -n 1 192.31.25.1; Start-Sleep -Seconds 2 }
```

Question 10 : Option deadline (-w)

Ubuntu :

```
ping -w2 192.31.25.1
```

Envoie des pings pendant 2 secondes puis s'arrête automatiquement.

Kali : Idem Ubuntu

```
ping -w2 192.31.25.1
```

Mac : Utiliser `-t` (différent de Linux)

```
ping -t 2 192.31.25.1
```

Windows : Utiliser `-w` pour le timeout par requête (pas la durée totale)

```
ping -w 2000 192.31.25.1
```

Note : `-w` sous Windows = timeout en ms par requête, pas durée totale.

Pour une durée limitée sous Windows, utiliser PowerShell ou une boucle avec timeout.

Question 11 : MTU et taille des pings

Vérification du MTU

```
ip a
```

Résultats attendus :

- MTU interface Ethernet : **1500 octets**
- MTU boucle locale (lo) : **65536 octets**

Analyse ping de taille 1

```
ping -c1 -s1 192.31.25.1
```

Analyse Wireshark :

- **Taille paquet IPv4** : 29 octets (20 IP + 8 ICMP + 1 data)
- **Taille ping (ICMP)** : 9 octets (8 en-tête + 1 data)
- **Taille données ICMP** : 1 octet
- **Contenus donnés** : 0x00 (ou autre valeur de remplissage)
- **Contenu réponse** : Echo Reply avec la même donnée (0x00)

Ping de taille 2

```
ping -c1 -s2 192.31.25.1
```

- **Taille paquet IPv4** : 30 octets
- **Taille ICMP** : 10 octets
- **Données ICMP** : 2 octets (ex: 0x00 00)

Ping de taille 3

```
ping -c1 -s3 192.31.25.1
```

- **Taille paquet IPv4** : 31 octets
- **Taille ICMP** : 11 octets
- **Données ICMP** : 3 octets (ex: 0x00 00 00)

Ping dépassant le MTU (5000 octets)

```
ping -c1 -s5000 192.31.25.1
```

Ce qui se passe : FRAGMENTATION IPv4

Le paquet est trop grand pour le MTU (1500 octets), il est donc fragmenté en plusieurs paquets IPv4.

Analyse dans Wireshark :

Fragment 1 :

- Identification : 0XXXXX (même ID pour tous les fragments)
- Flags : DF=0, **MF=1** (More Fragments = il y a d'autres fragments)
- Fragment Offset : **0** (premier fragment)
- Taille : 1500 octets (MTU)

Fragment 2 :

- Identification : 0XXXXX (même ID)
- Flags : DF=0, **MF=1**
- Fragment Offset : **185** (en unités de 8 octets → $185 \times 8 = 1480$ octets)
- Taille : 1500 octets

Fragment 3 :

- Identification : 0XXXXX (même ID)
- Flags : DF=0, **MF=1**
- Fragment Offset : **370** ($370 \times 8 = 2960$ octets)
- Taille : 1500 octets

Fragment 4 (dernier) :

- Identification : 0XXXXX
- Flags : DF=0, **MF=0** (dernier fragment)
- Fragment Offset : **555** ($555 \times 8 = 4440$ octets)
- Taille : reste des données

Calcul du Fragment Offset : La valeur affichée est en unités de 8 octets. Pour obtenir la position réelle en octets :

Position réelle = Valeur Fragment Offset × 8

Exemple : Fragment Offset = 185 → Position = 185 × 8 = 1480 octets

Question 12 : Option TTL

Ping avec TTL=1 vers la passerelle

```
ping -c1 -t1 192.31.25.1
```

Ce qui se passe :

-  **Le ping fonctionne** car la passerelle est à 1 saut
- TTL du paquet envoyé : **1**
- TTL de la réponse (Echo Reply) : **64** (valeur par défaut de la RT-Box2/Linux)

Capture Wireshark :

- Echo Request : TTL = 1
- Echo Reply : TTL = 64

Ping avec TTL=1 vers iut-rt

```
ping -c1 -t1 172.31.25.9
```

Ce qui se passe :

-  **Le ping échoue**
- **Raison :** iut-rt est sur le réseau 172.31.16.0/20, donc de l'autre côté de la passerelle (2 sauts nécessaires)
- Avec TTL=1, le paquet n'a droit qu'à 1 saut
- La passerelle décrémente le TTL de 1→0 et rejette le paquet

Message d'erreur reçu :

Time to live exceeded

Message ICMP reçu :

- **Type : 11** (Time Exceeded)
- **Code : 0** (Time to Live exceeded in Transit)

Analyse Wireshark :

1. Echo Request envoyé (TTL=1) vers 172.31.25.9
2. La passerelle décrémente TTL (1→0)
3. Passerelle envoie un **ICMP Type 11, Code 0** (TTL exceeded)

Valeur minimale de TTL nécessaire :

- **TTL = 2** (minimum pour atteindre iut-rt)
 - 1 saut : Ubuntu → RT-Box2
 - 2 sauts : RT-Box2 → iut-rt

Pour vérifier :

```
ping -c1 -t2 172.31.25.9
```

Synthèse des options TTL selon les OS

OS	Option pour changer le TTL	Exemple
Ubuntu/Kali	ping -t [ttl] [@IP]	ping -t 10 @IP
Mac	ping -m [ttl] [@IP]	ping -m 10 @IP
Windows	ping -i [ttl] [@IP]	ping -i 10 @IP

Voici un **résumé des commandes importantes du TP n°1 (ICMP)**, avec **une phrase claire d'explication pour chacune**, dans le même esprit que pour le TP ARP.

C'est idéal pour une **fiche de révision CTP**.

RÉSUMÉ DES COMMANDES IMPORTANTES – TP ICMP

1. ip a
2. Affiche la configuration réseau sous Linux (adresses IP, interfaces, MTU), utile pour vérifier que la machine est bien configurée.
3. ifconfig

Affiche la configuration réseau sous macOS, équivalent de ip a sous Linux.

4. ipconfig

Affiche la configuration réseau sous Windows (adresse IP, passerelle, masque).

4. ping
Teste la connectivité réseau entre deux machines en envoyant des requêtes ICMP Echo Request.
5. ping -c 1
Envoie un seul ping afin de faciliter l'analyse d'une unique trame dans Wireshark.
6. ping -n 1 (Windows)
Équivalent Windows de -c, permet de limiter le nombre de pings envoyés.
7. ping -i 2
Envoie un ping toutes les 2 secondes (sous Linux / macOS).

8. `sudo ping -i 0.1`
Réduit fortement l'intervalle entre les pings (nécessite les droits administrateur).
9. `sudo ping -f`
Envoie des pings le plus rapidement possible (flood ping), utilisé pour tester la charge réseau.
10. `ping -t` (Windows)
Envoie des pings en continu jusqu'à interruption manuelle (Ctrl+C).
11. `ping -w 2` (Linux)
Envoie des pings pendant une durée maximale de 2 secondes puis s'arrête automatiquement.
12. `ping -t 2` (Mac)
Limite la durée du ping à 2 secondes sous macOS (option différente de Linux).
13. `ping -w 2000` (Windows)
Définit le timeout d'une requête ping à 2000 ms (ce n'est pas une durée totale).
14. `ping -s 1`
Modifie la taille des données ICMP pour analyser l'impact sur la taille du paquet.
15. `ping -s 5000`
Envoie un ping dépassant le MTU afin d'observer la fragmentation IPv4.
16. `ping -t 1` (Linux)
Force une valeur TTL spécifique pour tester le nombre de sauts nécessaires jusqu'à la destination.
17. `ping -m 1` (Mac)
Modifie le TTL sous macOS (option différente de Linux).
18. `ping -i 1` (Windows)
Modifie le TTL sous Windows (attention : `-i` ≠ intervalle sur Windows).
19. `tracert`
Affiche la liste des routeurs traversés par un paquet IP jusqu'à la destination.
20. Wireshark – filtre `icmp`
Permet d'afficher uniquement les paquets ICMP pour analyser les pings.
21. Wireshark – Follow → Stream
Permet de reconstituer les données transportées dans les paquets ICMP (utile pour les CTF).

22. Ctrl + C

Interrompt une commande ping en cours d'exécution.

À retenir absolument pour l'examen

- ping = test de connectivité + analyse ICMP
- TTL différent selon l'OS → **OS Fingerprinting**
- -s permet d'étudier la **fragmentation IPv4**
- Options ping **diffèrent selon le système** (Linux / Mac / Windows)